

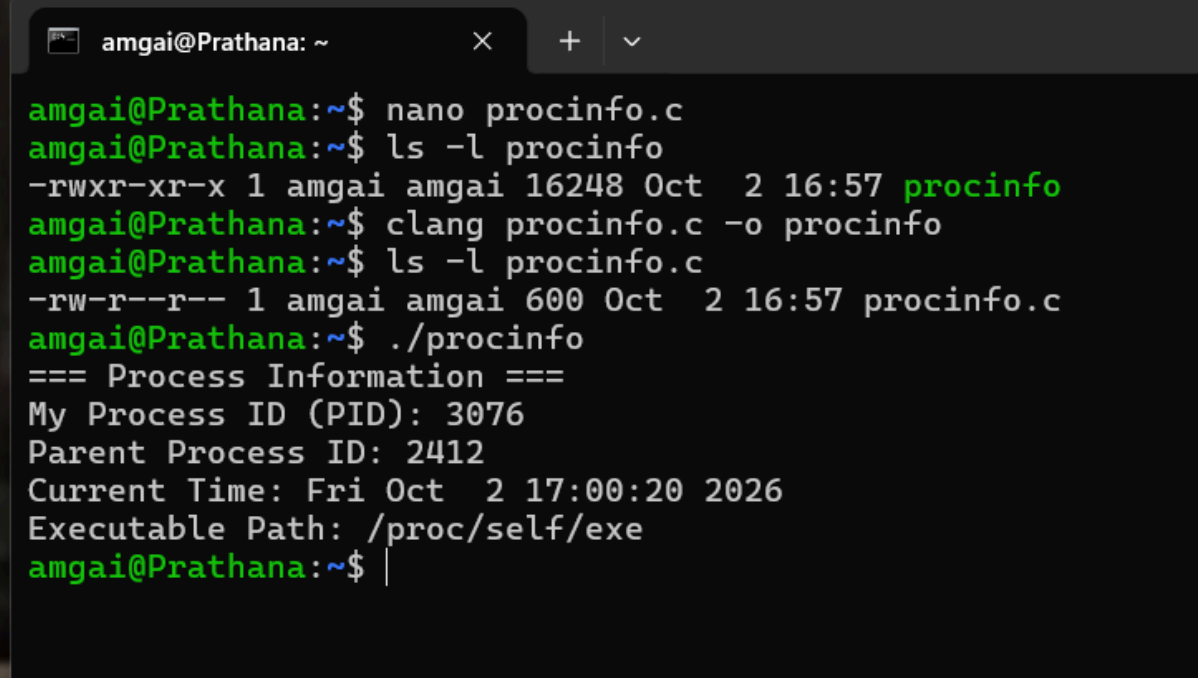
TABLE OF CONTENTS

III) A. THE C STANDARD LIBRARY: TWO HALVES	4
B. THE C STANDARD LIBRARY: TWO HALVES	4
IV. STATIC VS. DYNAMIC LINKING	5
V.) A .INSPECTING EXECUTABLES WITH READelf	5
B. INSPECTING EXECUTABLES WITH READelf	6
V. INSPECTING EXECUTABLES WITH READelf	7
VI. TRACING DEPENDENCIES WITH LDD	7
VI. TRACING DEPENDENCIES WITH LDD	8
IX. Knowledge Check Questions	8

LIST OF FIGURES

Figure 1: Creation, compilation and execution of procinfo.c	3
Figure 2: C Standard Library header files in /usr/include/	3
Figure 3: printf() and getpid() declarations in header files	4
Figure 4: C library files (libc.so.6 and libc.a)	4
Figure 5: Comparison of static and dynamic executable sizes	5
Figure 6: ELF Header of procinfo_dynamic	5
Figure 7: Program Headers of procinfo_dynamic	6
Figure 8: Dynamic Section of procinfo_dynamic	7
Figure 9: Shared library dependencies of procinfo_dynamic and static linking check	7
Figure 10: Shared library dependencies of procinfo	8

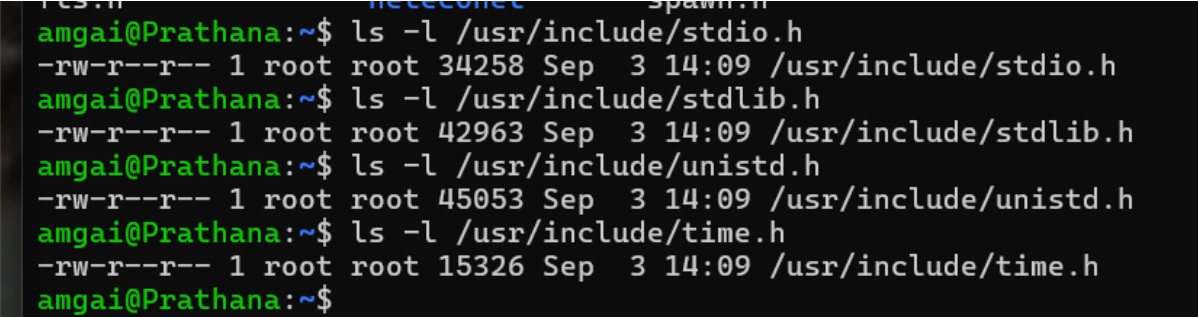
II. A PRACTICAL C PROGRAM: PROCESS INFORMATION



```
amgai@Prathana: ~  
amgai@Prathana:~$ nano procinfo.c  
amgai@Prathana:~$ ls -l procinfo  
-rwxr-xr-x 1 amgai amgai 16248 Oct  2 16:57 procinfo  
amgai@Prathana:~$ clang procinfo.c -o procinfo  
amgai@Prathana:~$ ls -l procinfo.c  
-rw-r--r-- 1 amgai amgai 600 Oct  2 16:57 procinfo.c  
amgai@Prathana:~$ ./procinfo  
=== Process Information ===  
My Process ID (PID): 3076  
Parent Process ID: 2412  
Current Time: Fri Oct  2 17:00:20 2026  
Executable Path: /proc/self/exe  
amgai@Prathana:~$ |
```

Figure 1: Creation, compilation and execution of `procinfo.c`

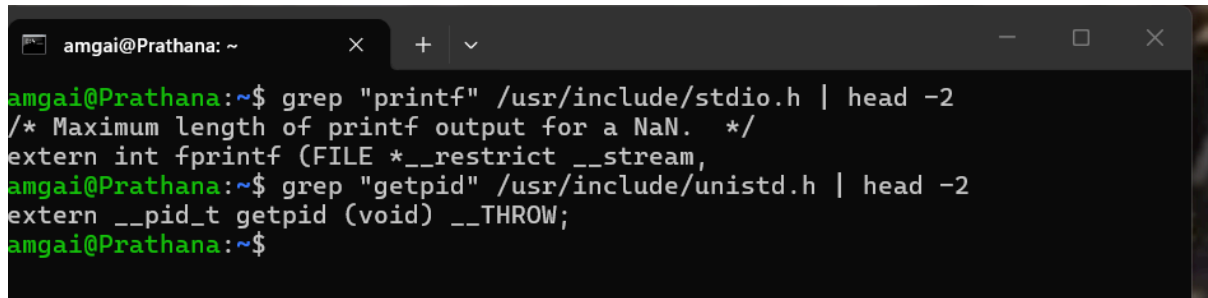
III. THE C STANDARD LIBRARY: TWO HALVES



```
amgai@Prathana:~$ ls -l /usr/include/stdio.h  
-rw-r--r-- 1 root root 34258 Sep  3 14:09 /usr/include/stdio.h  
amgai@Prathana:~$ ls -l /usr/include/stdlib.h  
-rw-r--r-- 1 root root 42963 Sep  3 14:09 /usr/include/stdlib.h  
amgai@Prathana:~$ ls -l /usr/include/unistd.h  
-rw-r--r-- 1 root root 45053 Sep  3 14:09 /usr/include/unistd.h  
amgai@Prathana:~$ ls -l /usr/include/time.h  
-rw-r--r-- 1 root root 15326 Sep  3 14:09 /usr/include/time.h  
amgai@Prathana:~$
```

Figure 2: C Standard Library header files in `/usr/include/`

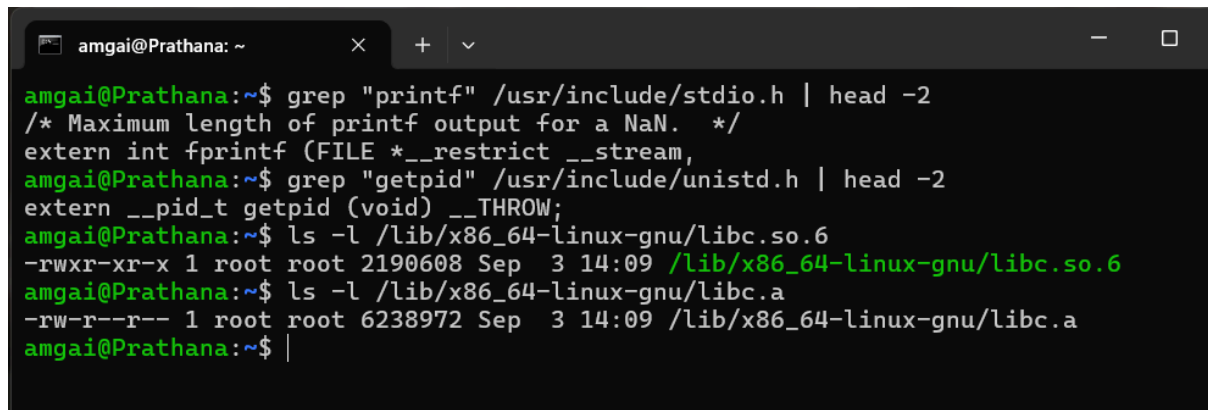
III) A. THE C STANDARD LIBRARY: TWO HALVES



```
amgai@Prathana: ~  
amgai@Prathana:~$ grep "printf" /usr/include/stdio.h | head -2  
/* Maximum length of printf output for a NaN. */  
extern int fprintf (FILE *__restrict __stream,  
amgai@Prathana:~$ grep "getpid" /usr/include/unistd.h | head -2  
extern __pid_t getpid (void) __THROW;  
amgai@Prathana:~$
```

Figure 3: `printf()` and `getpid()` declarations in header files

B. THE C STANDARD LIBRARY: TWO HALVES



```
amgai@Prathana: ~  
amgai@Prathana:~$ grep "printf" /usr/include/stdio.h | head -2  
/* Maximum length of printf output for a NaN. */  
extern int fprintf (FILE *__restrict __stream,  
amgai@Prathana:~$ grep "getpid" /usr/include/unistd.h | head -2  
extern __pid_t getpid (void) __THROW;  
amgai@Prathana:~$ ls -l /lib/x86_64-linux-gnu/libc.so.6  
-rwxr-xr-x 1 root root 2190608 Sep  3 14:09 /lib/x86_64-linux-gnu/libc.so.6  
amgai@Prathana:~$ ls -l /lib/x86_64-linux-gnu/libc.a  
-rw-r--r-- 1 root root 6238972 Sep  3 14:09 /lib/x86_64-linux-gnu/libc.a  
amgai@Prathana:~$ |
```

Figure 4: C library files (`libc.so.6` and `libc.a`)

IV. STATIC VS. DYNAMIC LINKING

```
amgai@Prathana:~$ gcc -static procinfo.c -o procinfo_static
ls -lh procinfo_static
-rwxr-xr-x 1 amgai amgai 961K Oct  3 07:45 procinfo_static
amgai@Prathana:~$ gcc procinfo.c -o procinfo_dynamic
ls -lh procinfo_dynamic
-rwxr-xr-x 1 amgai amgai 16K Oct  3 07:47 procinfo_dynamic
amgai@Prathana:~$ |
```

Figure 5: Comparison of static and dynamic executable sizes

V.) A .INSPECTING EXECUTABLES WITH READelf

```
amgai@Prathana:~$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x1140
  Start of program headers:              64 (bytes into file)
  Start of section headers:             14264 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:             14
  Size of section headers:               64 (bytes)
  Number of section headers:             31
  Section header string table index:     30
amgai@Prathana:~$ |
```

Figure 6: ELF Header of procinfo_dynamic

B. INSPECTING EXECUTABLES WITH READelf

```
amgai@Prathana: ~  
amgai@Prathana:~$ readelf -l procinfo_dynamic  
  
Elf file type is DYN (Position-Independent Executable file)  
Entry point 0x1140  
There are 14 program headers, starting at offset 64  
  
Program Headers:  
Type           Offset             VirtAddr           PhysAddr  
FileSiz        MemSiz             Flags             Align  
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040  
0x0000000000000310 0x0000000000000310 R               0x8  
INTERP         0x0000000000000374 0x0000000000000374 0x0000000000000374  
0x000000000000001c 0x000000000000001c R               0x1  
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]  
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000  
0x00000000000007b8 0x00000000000007b8 R               0x1000  
LOAD           0x0000000000000100 0x0000000000000100 0x0000000000000100  
0x0000000000000311 0x0000000000000311 R E             0x1000  
LOAD           0x0000000000000200 0x0000000000000200 0x0000000000000200  
0x00000000000001c0 0x00000000000001c0 R               0x1000  
LOAD           0x00000000000002d8 0x00000000000002d8 0x00000000000002d8  
0x0000000000000290 0x0000000000000290 RW              0x1000  
DYNAMIC        0x00000000000002d0 0x00000000000002d0 0x00000000000002d0  
0x00000000000001f0 0x00000000000001f0 RW              0x8  
NOTE           0x0000000000000350 0x0000000000000350 0x0000000000000350  
0x0000000000000024 0x0000000000000024 R               0x4  
NOTE           0x0000000000000217 0x0000000000000217 0x0000000000000217  
0x0000000000000030 0x0000000000000030 R               0x8  
NOTE           0x000000000000021a 0x000000000000021a 0x000000000000021a  
0x0000000000000020 0x0000000000000020 R               0x4  
GNU_PROPERTY   0x0000000000000217 0x0000000000000217 0x0000000000000217  
0x0000000000000030 0x0000000000000030 R               0x8  
GNU_EH_FRAME   0x0000000000000208 0x0000000000000208 0x0000000000000208  
0x0000000000000034 0x0000000000000034 R               0x4  
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000  
0x0000000000000000 0x0000000000000000 RW              0x10  
GNU_RELRO      0x00000000000002d8 0x00000000000002d8 0x00000000000002d8  
0x0000000000000280 0x0000000000000280 R               0x1  
  
Section to Segment mapping:
```

Figure 7: Program Headers of procinfo_dynamic

V. INSPECTING EXECUTABLES WITH READelf

```
amgai@Prathana:~$ readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
   Tag               Type              Name/Value
0x0000000000000001 (NEEDED)          Shared library: [libc.so.6]
0x000000000000000c (INIT)            0x1000
0x000000000000000d (FINI)            0x1304
0x0000000000000019 (INIT_ARRAY)      0x3d80
0x000000000000001b (INIT_ARRAYSZ)    8 (bytes)
0x000000000000001a (FINI_ARRAY)      0x3d88
0x000000000000001c (FINI_ARRAYSZ)    8 (bytes)
0x000000006ffffef5 (GNU_HASH)        0x390
0x0000000000000005 (STRTAB)          0x508
0x0000000000000006 (SYMTAB)          0x3b8
0x000000000000000a (STRSZ)          208 (bytes)
0x000000000000000b (SYMENT)          24 (bytes)
0x0000000000000015 (DEBUG)            0x0
0x0000000000000003 (PLTGOT)          0x3f80
0x0000000000000002 (PLTRELSZ)        192 (bytes)
0x0000000000000014 (PLTREL)          RELA
0x0000000000000017 (JMPREL)          0x6f8
0x0000000000000007 (RELA)            0x638
0x0000000000000008 (RELASZ)          192 (bytes)
0x0000000000000009 (RELAENT)         24 (bytes)
0x000000000000001e (FLAGS)           BIND_NOW
0x000000006ffffffb (FLAGS_1)         Flags: NOW PIE
0x000000006ffffffe (VERNEED)         0x5f8
0x000000006fffffff (VERNEEDNUM)      1
0x000000006ffffff0 (VERSYM)          0x5d8
0x000000006ffffff9 (RELACOUNT)        3
0x0000000000000000 (NULL)            0x0
amgai@Prathana:~$ |
```

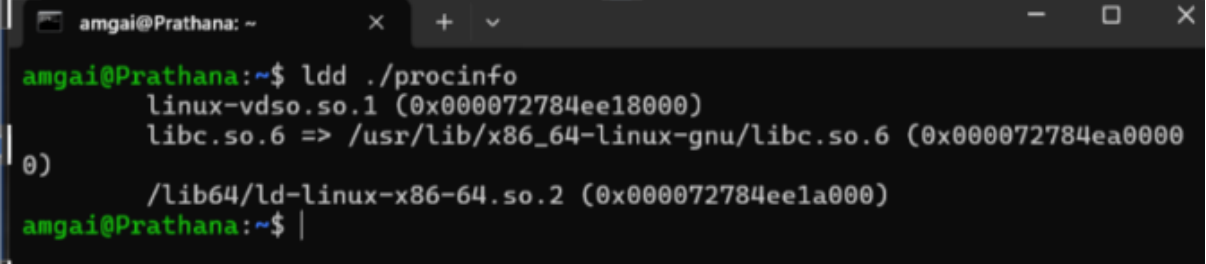
Figure 8: Dynamic Section of procinfo_dynamic

VI. TRACING DEPENDENCIES WITH LDD

```
amgai@Prathana:~$ ldd procinfo_dynamic
ldd procinfo_static
    linux-vdso.so.1 (0x000078490b987000)
    libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x000078490b600000)
    /lib64/ld-linux-x86-64.so.2 (0x000078490b989000)
not a dynamic executable
amgai@Prathana:~$ |
```

Figure 9: Shared library dependencies of procinfo_dynamic and static linking check

VI. TRACING DEPENDENCIES WITH LDD



```
amgai@Prathana: ~  
amgai@Prathana:~$ ldd ./procinfo  
linux-vdso.so.1 (0x000072784ee18000)  
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x000072784ea00000)  
0)  
/lib64/ld-linux-x86-64.so.2 (0x000072784ee1a000)  
amgai@Prathana:~$ |
```

Figure 10: Shared library dependencies of procinfo

IX. Knowledge Check Questions

1) What is the fundamental difference between a C header file and a shared library?

Answer: A header file contains function declarations, while a shared library contains the actual compiled code.

2) What is static and dynamic linking?

Answer:

- Static: Library code is included in the executable.
- Dynamic: Library code is loaded at runtime.
-

3) What is ELF Header? Write down the commands to check headers?

Answer: An ELF Header contains information about an executable file.

Commands:

```
readelf -h procinfo_dynamic  
readelf -l procinfo_dynamic  
readelf -d procinfo_dynamic
```

4) What does bookmark mean? provide an example.

Answer: A bookmark is a saved link or location that helps you quickly return to a specific webpage or place.

Example: Saving a website in your browser as a bookmark so you can open it later.